



Sistema de Gestión de Biblioteca Inteligente

Proyecto de Programación Orientada a Objetos

Materia

Programación Orientada a Objetos

Integrantes

Ange Benoit, Joan Balbastre, Hector Palomino, Alejandro Pelecha

Grado en Ingeniería Informática - Primer Curso

Introducción



Problema

Las bibliotecas pequeñas todavía usan métodos manuales para gestionar libros, usuarios y préstamos. Esto genera:

- Pérdida de información
- Dificultad para buscar materiales
- Control ineficiente de préstamos



Objetivo

Desarrollar un sistema de gestión que permita:

- Administrar materiales y usuarios
- Controlar préstamos y devoluciones
- Guardar datos de forma persistente

Solución: Un sistema digital automatizado que aplique principios de Programación Orientada a Objetos.

Programación Orientada a Objetos

Una forma de organizar el código pensando en "cosas" del mundo real



Clases

Plantillas para crear objetos



Objetos

Instancias con datos reales



Herencia

Reutilizar código padre



Encapsulación

Proteger datos internos



¿Por qué POO?

Nos permite organizar mejor el código, reutilizarlo y hacerlo más fácil de mantener. Ideal para proyectos que pueden crecer y evolucionar con el tiempo.

Arquitectura del Proyecto

Estructura de Carpetas

biblioteca-inteligente/

models/

services/

persistence/

ui/

utils/

<> main.py

- **models/**
Libro, Usuario, Prestamo, Material
- **services/**
Lógica de préstamos y reservas
- **persistence/**
Guardar y cargar datos JSON
- **ui/**
Ventanas y componentes visuales
- **utils/**
Funciones auxiliares y helpers

Arquitectura modular: Separación clara de responsabilidades para mejor mantenimiento y escalabilidad.

Clases Principales

Material

Clase padre

Atributos:

id, titulo, autor,
disponible

Libro

Hereda de Material

Atributos:

isbn, genero

Usuario

Atributos:

id, nombre,
prestamos[]

Prestamo

Atributos:

id, fecha_inicio,
fecha_fin, activo

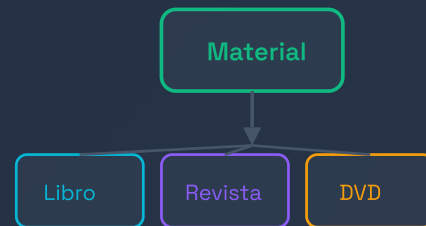
Relaciones entre clases



Un usuario puede tener varios préstamos. Cada préstamo está asociado a un material.

Herencia y Polimorfismo

Herencia



Todos heredan: id, título, autor

```
# Libro hereda de Material
class Libro(Material):
    def __init__(self, titulo, isbn):
        super().__init__(titulo)
        self.isbn = isbn
```

Polimorfismo

El mismo método funciona diferente según el tipo de objeto:

```
class Material:
    def descripcion_corta(self):
        return f"{self.titulo}"
```

```
class Libro(Material):
    def descripcion_corta(self):
        return f"{self.titulo} - ISBN: {self.isbn}"
```

```
class DVD(Material):
    def descripcion_corta(self):
        return f"{self.titulo} - Duración: {self.duracion}"
```

Cada tipo muestra su información de forma distinta.

Servicios y Lógica del Sistema



PrestamoService

- crear_prestamo()
- finalizar_prestamo()
- listar_activos()
- verificar_disponibilidad()



ReservaService

- crear_reserva()
- cancelar_reserva()
- convertir_a_prestamo()



UsuarioService

- registrar_usuario()
- buscar_usuario()
- historial_prestamos()

Ejemplo: Crear un préstamo



Persistencia de Datos

Cómo funciona



- Al iniciar: cargamos datos del JSON
- Al modificar: guardamos cambios

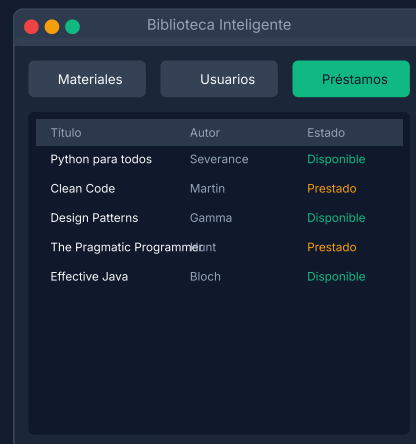
Ejemplo de archivo JSON

```
[
  {
    "id": "L001",
    "titulo": "Python
para todos",
    "autor": "Charles
Severance",
    "isbn": "978-
8494423570",
    "disponible": true
  }
]
```


Interfaz Gráfica con Tkinter

Tkinter es la biblioteca estándar de Python para crear interfaces gráficas. Viene instalada por defecto.

Ventana principal



Ventanas implementadas

- Listado de materiales
- Registro de usuarios
- Gestión de préstamos
- Búsqueda avanzada
- Devoluciones

Componentes usados

Frame

Label

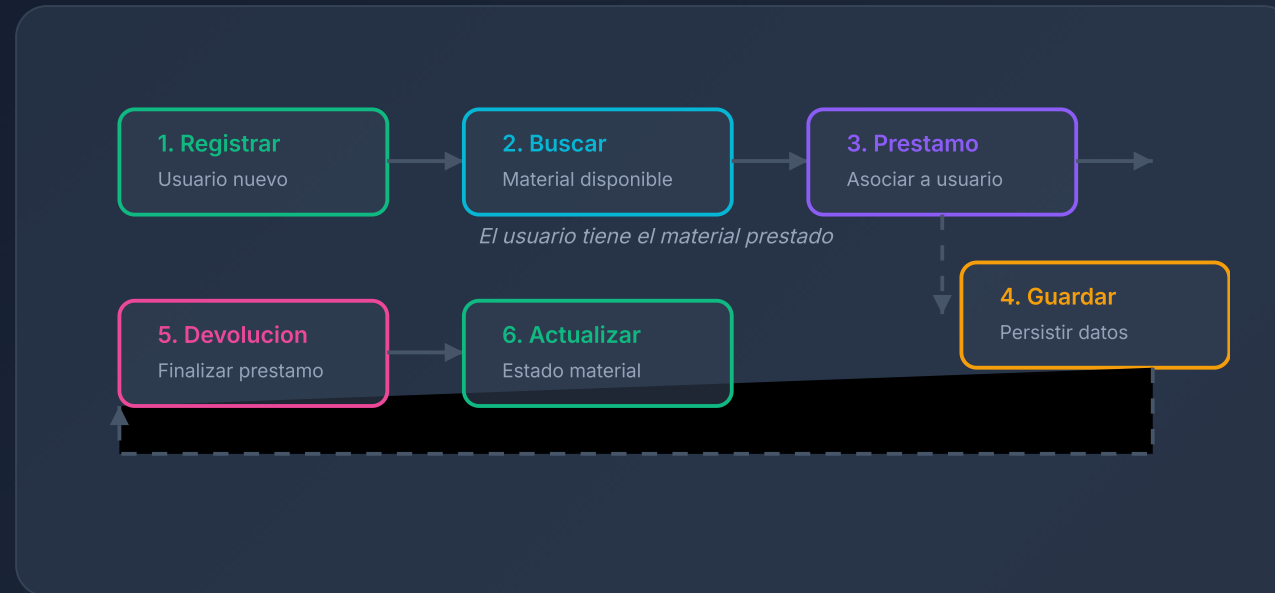
Button

Entry

Treeview

Messagebox

Flujo Completo



4

Pasos para prestar

2

Pasos para devolver

JSON

Datos persistentes

Problemas Encontrados



Organización de archivos

Al principio no sabíamos bien cómo dividir las responsabilidades. Solucionamos esto investigando sobre arquitectura por capas.



Manejo de errores

Al principio no controlábamos excepciones. Agregamos bloques try-except y mensajes al usuario.



Comunicación entre clases

Costó entender cómo pasar datos entre servicios y modelos. Usamos inyección de dependencias simple.



Relaciones entre objetos

Diseñar cómo se relacionan Usuario, Préstamo y Material fue complejo. Hicimos varios diagramas antes.



Persistencia JSON

Convertir objetos a diccionarios y viceversa fue tedioso. Creamos métodos `to_dict()` y `from_dict()`.



Lección aprendida

Planificar antes de programar ahorra mucho tiempo. Los diagramas UML nos ayudaron mucho.

Qué Aprendimos



Trabajo en equipo

- Usamos Git y GitHub para coordinar
- Dividimos tareas por módulos
- Hicimos code reviews entre nosotros



Programación Orientada a Objetos

- Entendimos herencia y polimorfismo
- Aprendimos a encapsular datos
- Separamos responsabilidades



Organización del código

- Estructurar proyectos en carpetas
- Importar módulos correctamente
- Escribir código más legible



Diseño de software

- Pensar antes de programar
- Usar diagramas UML
- Planificar arquitectura

Mejoras Futuras



Base de datos SQL

Cambiar JSON por SQLite o PostgreSQL para mejor rendimiento.

Alta prioridad



Login de usuarios

Sistema de autenticación con roles: admin, bibliotecario, lector.

Media prioridad



Mejor interfaz

Usar PyQt o interfaz web con Flask/Django.

Media prioridad



Sistema de multas

Calcular y gestionar multas por retrasos.

Baja prioridad



Estadísticas

Reportes de libros más prestados, usuarios activos.

Baja prioridad



API REST

Crear API para que otras apps se conecten.

Futuro

Conclusión

Desarrollamos un sistema funcional que aplica los conceptos de POO de forma práctica. Aprendimos que **organizar bien el código desde el principio** ahorra muchos problemas después.

5

Clases
creadas

15+

Funcionalidades

4

Integrantes

100%

Python

Logros principales

POO aplicada

Persistencia JSON

Interfaz gráfica

Código modular



Preguntas

Gracias por vuestra atención

Repositorio del proyecto

<https://github.com/ByJoan7/Biblioteca>

Ange Benoit, Joan Balbastre, Hector Palomino, Alejandro Pelecha

Programación Orientada a Objetos - Primer Curso